UNIDAD 3

DISEÑO Y REALIZACIÓN DE PRUEBAS

IES Francisco Ayala

Ciclo Formativo de Grado Medio: ***Desarrollo de Aplicaciones Web***

Módulo: **ENTORNOS DE DESARROLLO**

Contenidos

1. Filosofía de las pruebas del software
2. Estrategia de pruebas del software
3. Técnicas de diseño de casos de prueba
4. Documentación de las pruebas
5. Herramientas de depuración
6. Pruebas automáticas

3.2. Pruebas de caja negra

3. Técnicas de diseño de casos de prueba. Pruebas de caja negra

El objetivo de las pruebas de caja negra es **crear casos de prueba que permitan revisar todos los requisitos funcionales** de un programa.

Para ello, existen **distintas técnicas** de diseño de casos de prueba de caja negra.

3. Técnicas de diseño de casos de prueba. Pruebas de caja negra

PARTICIONES O CLASES DE EQUIVALENCIA

- En esta técnica se identifican un conjunto de **clases de equivalencia o tipos de valores** que se deben asignar a los campos de entrada.
- Por ejemplo, supongamos un campo de entrada llamado *número de empleado*, definido con una serie de condiciones: numérico de 3 dígitos y el primero no puede ser 0. Entonces se puede definir una clase de equivalencia no válida: *número de empleado < 100*; y otra válida: *número de empleado entre 100 y 999*.

3. Técnicas de diseño de casos de prueba. Pruebas de caja negra

PARTICIONES O CLASES DE EQUIVALENCIA

- Para identificar las clases de equivalencia se examina cada **condición de entrada** (son parte del dominio de valores de la entrada y normalmente son una frase en la especificación) y se divide en **dos o más grupos**. Se definen dos tipos de clases de equivalencia:
  - **Clases válidas**: son los valores de entrada válidos.
  - **Clases no válidas**: son los valores de entrada no válidos.

3. Técnicas de diseño de casos de prueba. Pruebas de caja negra

PARTICIONES O CLASES DE EQUIVALENCIA

En la siguiente tabla se resume el número de clases de equivalencia válidas y no válidas que hay que definir para cada tipo de condición de entrada:

Condiciones de entrada	Nº de Clases de equivalencia válidas	Nº de Clases de equivalencia no válidas
1 . Rango	1 CLASE VÁLIDA Contempla los valores del rango	2 CLASES NO VÁLIDAS Un valor por encima del rango Un valor por debajo del rango
2. Valor específico	1 CLASE VÁLIDA Contempla dicho valor	2 CLASES NO VÁLIDAS Un valor por encima Un valor por debajo
3. Miembro de un conjunto	1 CLASE VÁLIDA Una clase por cada uno de los miembros del conjunto	1 CLASE NO VÁLIDA Un valor que no pertenece al conjunto
4. Lógica	1 CLASE VÁLIDA Una clase que cumpla la condición	1 CLASE NO VÁLIDA Una clase que no cumpla la condición

EJERCICIO
(CLASES DE EQUIVALENCIA)

EJERCICIO: CLASES DE EQUIVALENCIA

En una aplicación, la persona usuaria debe introducir su edad, que debe ser un número entre 18 y 65; su NIF, que debe constar de 8 números y una letra; y, además, debe indicar si tiene o no nacionalidad española, de forma que tener nacionalidad española es un requisito.

SOLUCIÓN DEL EJERCICIO: CLASES DE EQUIVALENCIA

Tabla 3.1. Condiciones de entrada para el supuesto, junto con las clases de equivalencia válidas y no válidas de acuerdo con las normas especificadas

Condición de entrada	Clases válidas	Clases no válidas
Edad	18 ≤ edad ≤ 65 (1)	edad < 18 (2) edad > 65 (3) No es un número (4)
NIF	Una cadena de 9 caracteres compuesta por 8 números y una letra (5)	< 9 caracteres (6) > 9 caracteres (7) Alguno de los 8 primeros caracteres no es un número (8) El último carácter no es una letra (9)
Nacionalidad	Española (10)	No española (11)

En la Tabla 3.2, se muestran los casos de prueba válidos y, en la Tabla 3.3, los no válidos. Por cada caso de prueba, se indica, en la última columna, las clases de equivalencia incluidas. Recuerda que, para el caso de las clases válidas, se debe crear el número mínimo de casos de prueba que incluyan todas las clases válidas; en este caso, se requiere un único caso de prueba.

Tabla 3.2. Caso de prueba con clases de equivalencia válidas

Edad	NIF	Nacionalidad	Clases incluidas
35	32323267G	Española	(1), (5), (10)

Para las clases no válidas, hay que crear un caso de prueba por cada una de ellas. Como se puede observar en la Tabla 3.1, al haber 8 clases de equivalencia no válidas, se deben crear 8 casos de prueba (Tabla 3.3).

Tabla 3.3. Casos de prueba con clases de equivalencia no válidas

Edad	NIF	Nacionalidad	Clases incluidas
16	78787654Z	Española	(2), (5), (10)
73	88788888U	Española	(3), (5), (10)
AB	56837483Y	Española	(4), (5), (10)
45	879847F	Española	(1), (6), (10)
23	6767676762[a]	Española	(1), (7), (10)
64	TT789009R	Española	(1), (8), (10)
19	569832349	Española	(1), (9), (10)
23	98828282C	No española	(1), (5), (11)

3. Técnicas de diseño de casos de prueba. Pruebas de caja negra

ANÁLISIS DE VALORES LÍMITE

El análisis de valores límite se basa en que los errores tienden a producirse con más probabilidad en los límites o extremos de los campos de entrada.

Esta técnica complementa a la anterior y los casos de prueba elegidos ejercitan los valores justo por encima y por debajo de los márgenes de la clase de equivalencia. Además, no solo se centra en las condiciones de entrada, sino que también se exploran las condiciones de salida definiendo las clases de equivalencia de salida.

3. Técnicas de diseño de casos de prueba. Pruebas de caja negra

ANÁLISIS DE VALORES LÍMITE

Las reglas son las siguientes:

- Si una condición de entrada especifica un ***rango de valores***, se deben diseñar casos de prueba para los límites del rango y para los valores justo por encima y por debajo del rango. Por ejemplo, si una entrada requiere un rango de valores enteros comprendidos entre 1 y 10, hay que escribir casos de prueba para el valor 1, 10, 0 y 11.

- Si una condición de entrada especifica un ***número de valores***, se deben diseñar casos de prueba que ejerciten los valores máximo, mínimo, un valor justo por encima del máximo y un valor justo por debajo del mínimo. Por ejemplo, si el programa requiere de dos a diez datos de entradas, hay que escribir casos de prueba para 2, 10, 1 y 11 datos de entrada.

3. Técnicas de diseño de casos de prueba. Pruebas de caja negra

ANÁLISIS DE VALORES LÍMITE

Las reglas son las siguientes:

- Aplicar la regla 1 para la condición de salida. Por ejemplo, si se debe aplicar sobre un campo de salida un descuento de entre un 10% mínimo y un 50% máximo (dependiendo del tipo de cliente); se generarán casos de prueba para 9.99%, 10%, 50% y 50.01%.

- Usar la regla 2 para la condición de salida. Por ejemplo, si la salida de un programa es una tabla de temperaturas de 1 a 10 elementos, se deben diseñar casos de prueba para que la salida del programa produzca 0, 1, 10 y 11 elementos. Tanto en esta regla, como en la anterior, hay que tener en cuenta que no siempre se podrán generar resultados fuera del rango de salida.

- Si las estructuras de datos internas tienen límites preestablecidos (por ejemplo un array de 100 elementos), hay que asegurarse de diseñar casos de prueba que ejercite la estructura de datos en sus límites, primer y último elemento.

EJEMPLO (ANÁLISIS DE VALORES LÍMITE)

Determina los casos de prueba para los siguientes elementos según las condiciones de entrada y de salida:

	Condiciones de entrada y salida	Casos de prueba
Código	Entero de 1 a 100	Valores: 0, 1, 100, 101
Puesto	Alfanumérico de hasta 4 caracteres	Longitud de caracteres: 0, 1, 4, 5
Antigüedad	De 0 a 25 años (Real)	Valores: 0, 25, -0.1, 25.1
Horas semanales	De 0 a 60	Valores: 0, 60, -1, 61
Fichero de entrada	Tiene de 1 a 100 registros	Para leer 0, 1, 100 y 101 registros
Fichero de salida	Podrá tener de 0 a 10 registros	Para generar 0, 10 y 11 registros (no se puede generar -1 registro)
Array interno	De 20 cadenas de caracteres	Para el primer y último elemento.

Entornos de desarrollo. Alicia Ramos, María Jesús Ramos. Garceta. 2014

EJERCICIO
(ANÁLISIS DE VALORES LÍMITE)

EJERCICIO: ANÁLISIS DE VALORES LÍMITE

Partiendo de la condición de entrada edad del ejercicio de las clases de equivalencia, indica qué clases de equivalencia adicionales habría que crear, y genera los casos de prueba correspondientes empleando la técnica de análisis de valores límite.

SOLUCIÓN DEL EJERCICIO: ANÁLISIS DE VALORES LÍMITE

Dado que esta condición de entrada especifica un número de valores (entre 18 y 65), hay que generar clases de equivalencia para los valores mínimo y máximo (18 y 65), uno menos que el mínimo (17) y uno más que el máximo (66), como se muestra en la Tabla 3.4, en la que se ha asignado a las clases de equivalencia una numeración que sigue la de la Actividad resuelta 3.3:

Tabla 3.4. Clases de equivalencia válidas y no válidas para la condición de entrada edad empleando la técnica de análisis de valores límite

Condición de entrada	Clases válidas	Clases no válidas
Edad	edad = 18 (12) edad = 65 (13)	edad = 17 (14) edad = 66 (15)

Una vez generada estas clases de equivalencia adicionales, se trata de generar los casos de prueba que abarquen estas clases. Para ello, se debe tener en cuenta que hay que generar un caso de prueba válido que abarque todas las clases válidas. Como, en este caso, para la edad, hay dos clases válidas, se deben generar dos casos de prueba válidos (véase Tabla 3.5). A la edad, se deben añadir las otras condiciones de entrada que se incluyeron en la Actividad resuelta 3.3.

Tabla 3.5. Casos de prueba con clases de equivalencia válidas que se generan empleando la técnica de análisis de valores límite

Edad	NIF	Nacionalidad	Clases incluidas
18	32323267G	Española	(12), (5), (10)
65	32323267G	Española	(13), (5), (10)

Hay que generar, además, un caso de prueba nuevo por cada clase no válida. Como se han añadido dos nuevas clases no válidas, se generará un caso de prueba no válido por cada una de ellas (Tabla 3.6).

Tabla 3.6. Casos de prueba con clases de equivalencia no válidas que se generan empleando la técnica de análisis de valores límite

Edad	NIF	Nacionalidad	Clases incluidas
17	32323267G	Española	(14), (5), (10)
66	32323267G	Española	(15), (5), (10)

3. Técnicas de diseño de casos de prueba. Pruebas de caja negra

CONJETURA DE ERRORES

Esta técnica consiste en generar una **lista de errores** que suelen cometer quienes desarrollan aplicaciones y de las **situaciones** que suelen dar lugar a dichos errores, para luego generar un conjunto de casos de prueba basándose en esta lista. Se trata de errores que se cometen comúnmente y no relacionados con aspectos funcionales.

3. Técnicas de diseño de casos de prueba. Pruebas de caja negra

CONJETURA DE ERRORES

Ejemplos típicos:

- El valor 0 tanto en la entrada como en la salida es una situación propensa a que se cometan errores.
- En situaciones en las que se introduce un número variable de valores, como por ejemplo una lista, conviene considerar los siguientes casos: no introducir ningún valor, introducir un solo valor y un conjunto de valores en el que todos son iguales.
- La usuaria o usuario de la aplicación puede introducir datos erróneos, por ejemplo, que en un campo numérico introduzca algún carácter no numérico.
- La persona encargada de la programación pueda haber interpretado incorrectamente algo en la especificación.

4. Documentación de las pruebas

4. Documentación de las pruebas

- La **documentación** de las pruebas es conveniente para una buena **organización** de estas y para asegurar su **reutilización**.

- Se deben documentar tanto el **diseño de las pruebas** como el **resultado o ejecución de las pruebas**.

4. Documentación de las pruebas

- **DOCUMENTACIÓN DEL DISEÑO DE LAS PRUEBAS**:
  - **Plan de pruebas**: documento con las planificación general de las pruebas para la aplicación que se está creando. Se debe indicar el enfoque general de las pruebas, los recursos requeridos, las actividades de pruebas que se van a llevar a cabo, los elementos que se deben probar, el personal responsable y los riesgos asociados.
  - **Especificación del diseño de las pruebas**: informe que detalla el plan de pruebas porque en él se indica cada uno de los elementos concretos que se van a probar y los procedimientos de prueba. A partir de este documento se generan los siguientes:
    - ***Especificaciones de casos de prueba***: por cada prueba que se va a realizar, se deben indicar los datos de entrada que se van a suministrar, los resultados esperados y las dependencias entre casos de prueba.
    - ***Especificaciones de procedimientos de prueba***: especifica los pasos para la ejecución de uno o de un conjunto de casos de prueba. En este documento, se deben indicar la lista de casos de prueba correspondientes al procedimiento, los requisitos para la ejecución de los casos de prueba (entorno, personal, etc.) y los pasos del procedimiento.

4. Documentación de las pruebas

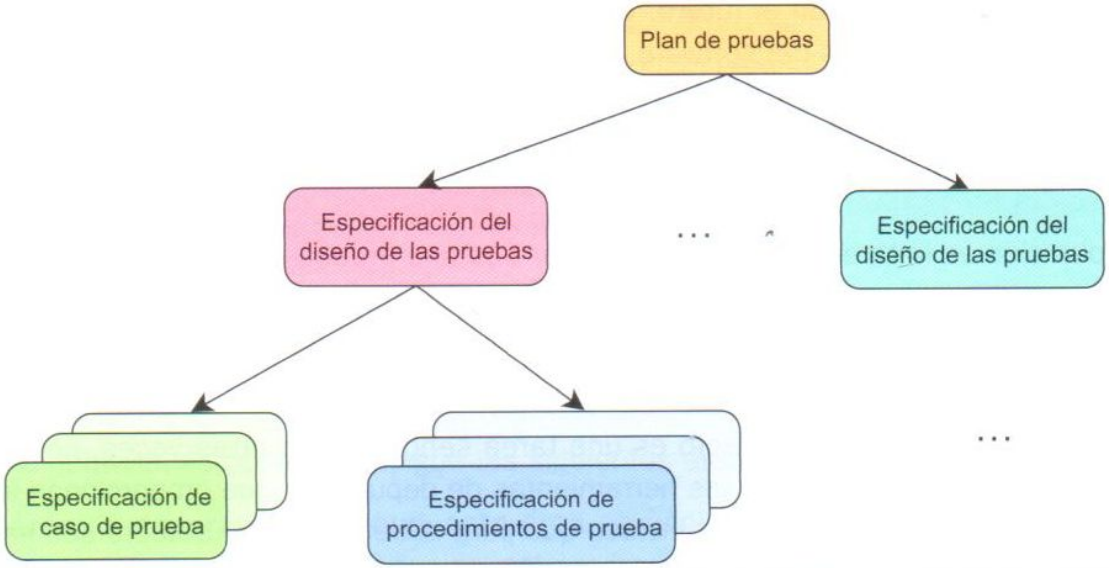
- **Documentación de la ejecución de las pruebas**: importante para la eficacia en la detección y corrección de defectos y para dejar constancia de los resultados de las pruebas. La ejecución de las pruebas toma como entrada las especificaciones de los casos de prueba que se van a usar y las especificaciones de los procedimientos de prueba, y debe generar dos tipos de documentos:

  - **Histórico de pruebas**: registra todos los hechos relevantes ocurridos durante la ejecución: elementos probados, entorno de pruebas, resultados obtenidos, fecha y hora y referencia al informe de incidentes, si es el caso.

  - **Informe de incidentes**: registra cada incidente ocurrido durante la prueba que requiera una posterior investigación.

  La ejecución de las pruebas genera asimismo un **informe resumen de pruebas**, que incluye: resumen de la evaluación de los elementos probados, variaciones del software como resultado de las pruebas, valoración de la cobertura de la prueba, resumen de los resultados obtenidos y resumen de las actividades de prueba (incluyendo el consumo de recursos).

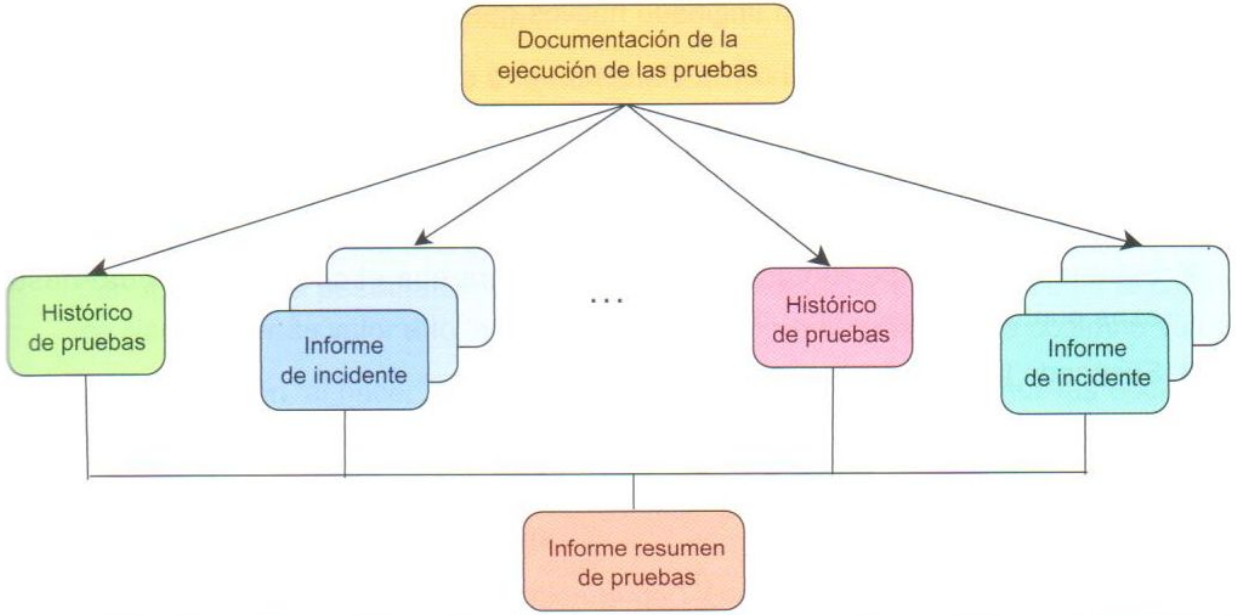
4. Documentación de las pruebas



Entornos de desarrollo. José Manuel Piñeiro Gómez. Paraninfo. 2022

El plan de pruebas es un documento único para una aplicación con indicaciones generales acerca de las pruebas. Este se desglosa, por cada elemento que se va a probar, en varias especificaciones del diseño de las pruebas. Cada uno de estos documentos contiene además varias especificaciones de casos de prueba y varias especificaciones de procedimientos de pruebas.

4. Documentación de las pruebas



Entornos de desarrollo. José Manuel Piñeiro Gómez. Paraninfo. 2022

Se parte de una especificación del diseño de las pruebas y, por cada ejecución de estas, a partir de una serie de casos de pruebas y sus procedimientos, se genera un histórico de pruebas con todos los hechos relevantes ocurridos durante esas pruebas y un informe por cada incidente ocurrido. Todo ello se recoge en el informe resumen de pruebas.

4. Documentación de las pruebas

- Es importante la valoración de los procesos de prueba porque puede ayudar en proyectos posteriores. Por ello, es conveniente realizar un **análisis causal**, cuyo objetivo es proporcionar información sobre la naturaleza de los defectos encontrados. **Por cada defecto** encontrado, se debería registrar la siguiente información: *cuándo se cometió, quién lo cometió, qué se hizo mal, cómo se podría haber evitado, por qué no se detectó antes, cómo se podría haber detectado antes y cómo se encontró el error*.

- El objetivo del análisis causal, además de proporcionar información sobre la naturaleza de los defectos encontrados, es formar al personal sobre los errores cometidos para que se puedan **prevenir en el futuro**. La información generada se puede emplear para predecir los fallos futuros del software.

5. Herramientas de depuración

5. Herramientas de depuración

La **depuración** es la tarea que hay que realizar tras una prueba exitosa, es decir, tras una prueba que detecta un fallo. En este caso, lo que se hace es localizar el defecto en el software y corregirlo.

A veces, la localización de un defecto es una tarea sencilla, pero otras veces puede resultar más complicada, y para ello, pueden ser útiles las **herramientas de depuración** que proporcionan los entornos de desarrollo.

Una vez detectado el defecto, habrá que corregirlo y tener en cuenta que al realizar las modificaciones para corregir el fallo, se pueden introducir nuevos defectos, por lo que será conveniente ejecutar pruebas de regresión, es decir, se repetirán los casos de prueba que se ejecutaron antes de las modificaciones para asegurarse de que los cambios no han originado nuevos defectos.

5. Herramientas de depuración

Un IDE incorpora un **depurador** o ***debugger*** proporcionando las siguientes ayudas:

- Ejecuta el código paso a paso.
- Establece puntos de ruptura o interrupción para que el código se ejecute hasta esos puntos sin ejecutarse paso a paso.
- Suspende la ejecución del programa.
- Examina los valores de las variables a lo largo de la ejecución.

5. Herramientas de depuración

Uso del **depurador de Eclipse**:

https://www.eclipse.org/community/eclipse_newsletter/2017/june/article1.php

https://youtu.be/ymV7lUUHkUU?si=0IRSxNg0ubqO906q

https://youtu.be/uevYBh9_uH4?si=cyH30Dk73hkzmjJG

Uso del **depurador de Netbeans**:

https://netbeans.apache.org/tutorial/main/kb/docs/php/debugging/#_using_the_debugger_toolbar

https://youtu.be/TGtBqHQim_o?si=H5AgiIdyWKT0Z_5A

https://youtu.be/xCvoSnR-PQg?si=wF9CiYTKf4uqj-FN

EJERCICIO: Ejemplo de uso del depurador de un IDE

Realiza un documento en el que deberás explicar (capturas de pantalla, vídeos, textos, etc.) el uso del depurador de un IDE con todas las ayudas que ofrece (áreas o paneles de depuración, ejecución paso a paso, puntos de ruptura, suspensión de la ejecución, valores de las variables, etc.). Para ello utilizarás el código dado en el fichero adjunto.

El formato del fichero a entregar debe ser ".pdf".

Una vez realizado, se deberá entregar a través de la tarea "*Tarea evaluable: Depurador de un IDE*" de la Unidad 3, en la plataforma Moodle Centros.

6. Pruebas automáticas

6. Pruebas automáticas

Las **pruebas de unidad o unitarias**, es decir, las pruebas que afectan a una clase y a sus métodos, implican crear algún objeto de la clase que se está probando y realizar llamadas a sus métodos. Esto se puede hacer manualmente o usando herramientas que permiten realizar pruebas unitarias de manera automatizada, como **JUnit**, integrada en el IDE Netbeans o Eclipse.

6. Pruebas automáticas

JUnit es una herramienta que permite realizar tests unitarios para comprobar el correcto funcionamiento de las clases que estamos desarrollando.

Uso de **JUnit en Eclipse**:

https://www.eclipse.org/community/eclipse_newsletter/2017/october/article5.php

https://youtu.be/wkXL3emg-NU?si=LzZ3Yp6Cu8aMxUiC

https://youtu.be/ql0BX4hq6D4?si=YlrMfwNYQPox9Y6d

Uso de **JUnit en Netbeans**:

https://netbeans.apache.org/tutorial/main/kb/docs/java/junit-intro/

https://youtu.be/4wdQ_Xdg_l0?si=R6ZlYY81XNWMtsrh

https://youtu.be/ZOGz_1XtTKc?si=32uZuXfKVDg_DJgK

EJERCICIO: Ejemplo de uso de la herramienta JUnit de un IDE

Realiza un documento en el que deberás explicar (capturas de pantalla, vídeos, textos, etc.) el uso de la herramienta JUnit de un IDE. Para ello utilizarás el código dado en el fichero adjunto.

El formato del fichero a entregar debe ser “.pdf”.

Una vez realizado, se deberá entregar a través de la tarea “*Tarea evaluable: JUnit de un IDE*” de la Unidad 3, en la plataforma Moodle Centros.